

MapReduce: Simplified Data Processing on Large Clusters

A Tutorial Based on Google's Seminal Paper

Jeffrey Dean and Sanjay Ghemawat

Google, Inc.

Presented at OSDI 2004



Tutorial based on the paper:
"MapReduce: Simplified Data Processing on Large Clusters"

OSDI 2004

Outline

Introduction and Motivation

Programming Model

Implementation

Refinements

Performance

Experience at Google

Key Learnings and Conclusions

The Problem at Google (circa 2004)

Challenges:

- ▶ Processing **terabytes** of data
- ▶ Hundreds of special-purpose computations
- ▶ Examples:
 - ▶ Crawled documents
 - ▶ Web request logs
 - ▶ Inverted indices
 - ▶ Graph structures
 - ▶ Page summaries

Issues:

- ▶ Input data is **huge**
- ▶ Must distribute across **hundreds/thousands** of machines
- ▶ Complex code for:
 - ▶ Parallelization
 - ▶ Data distribution
 - ▶ Fault tolerance
 - ▶ Load balancing

Core Problem

Simple computations obscured by large amounts of complex code dealing with distribution and fault tolerance

The MapReduce Solution

Key Insight

Most computations involve:

1. Applying a **map** operation to each logical record
2. Applying a **reduce** operation to all values sharing the same key

Inspiration: Map and reduce primitives from functional programming (Lisp, etc.)

What MapReduce Provides:

- ▶ Simple programming interface
- ▶ Automatic parallelization
- ▶ Fault tolerance via re-execution
- ▶ Load balancing

Benefits:

- ▶ Hides messy details
- ▶ Programmers focus on logic
- ▶ No distributed systems expertise needed
- ▶ Scales to thousands of machines

Major Contributions

1. Simple and Powerful Interface

- ▶ Enables automatic parallelization and distribution
- ▶ Easy to express large-scale computations

2. High-Performance Implementation

- ▶ Achieves high performance on large clusters of commodity PCs
- ▶ Processes many terabytes on thousands of machines

3. Proven at Scale

- ▶ Hundreds of MapReduce programs implemented at Google
- ▶ Thousands of jobs executed daily on Google's clusters

MapReduce Programming Model

Core Concept

Computation takes a set of **input** key/value pairs and produces a set of **output** key/value pairs

User specifies two functions:

1. Map Function

- ▶ Takes an input pair
- ▶ Produces a set of *intermediate* key/value pairs
- ▶ Library groups all intermediate values by the same intermediate key

2. Reduce Function

- ▶ Accepts an intermediate key and a set of values for that key
- ▶ Merges values to form a possibly smaller set
- ▶ Typically produces zero or one output value per invocation

Type Signatures

Conceptual Types

Even though implementations may use strings, conceptually:

Map: $(k_1, v_1) \rightarrow \text{list}(k_2, v_2)$

Reduce: $(k_2, \text{list}(v_2)) \rightarrow \text{list}(v_2)$

Important Notes:

- ▶ Input keys/values (k_1, v_1) are from a **different domain** than output keys/values
- ▶ Intermediate keys/values (k_2, v_2) are from the **same domain** as output keys/values
- ▶ Values supplied to Reduce via an **iterator** (handles large lists)

Classic Example: Word Count

Problem: Count occurrences of each word in a large collection of documents

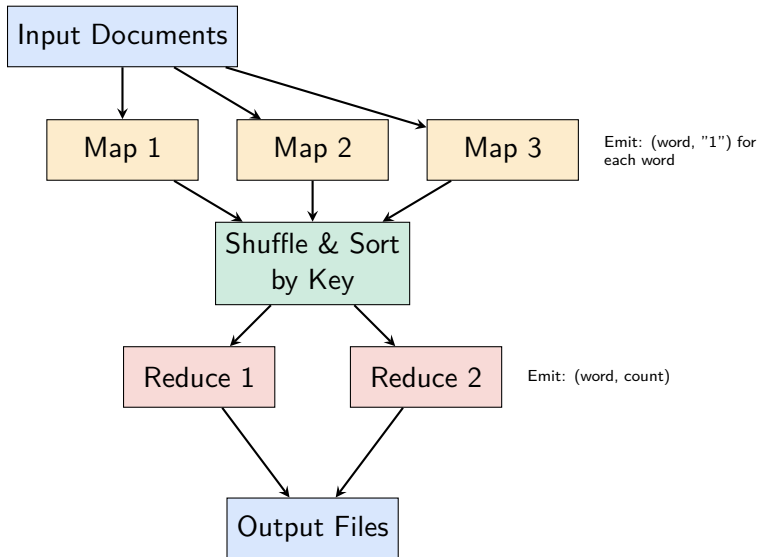
Listing 1: Map Function

```
1 map(String key, String value):  
2     // key: document name  
3     // value: document contents  
4     for each word w in value:  
5         EmitIntermediate(w, "1");
```

Listing 2: Reduce Function

```
1 reduce(String key, Iterator values):  
2     // key: a word  
3     // values: a list of counts  
4     int result = 0;  
5     for each v in values:  
6         result += ParseInt(v);  
7     Emit(AsString(result));
```


Word Count: Execution Flow



More Examples

Distributed Grep

Map: Emit line if it matches pattern

Reduce: Identity function (copies data to output)

Count of URL Access Frequency

Map: Process web logs, output $\langle \text{URL}, 1 \rangle$

Reduce: Add all values for same URL, emit $\langle \text{URL}, \text{total count} \rangle$

Reverse Web-Link Graph

Map: For each link to target in page source, output $\langle \text{target}, \text{source} \rangle$

Reduce: Concatenate all sources, emit $\langle \text{target}, \text{list}(\text{source}) \rangle$

Inverted Index

Map: Parse document, emit $\langle \text{word}, \text{document ID} \rangle$

Reduce: Sort document IDs, emit $\langle \text{word}, \text{list}(\text{document ID}) \rangle$

Google's Computing Environment

Target Platform: Large clusters of commodity PCs connected by switched Ethernet

Hardware Specs:

- ▶ Dual-processor x86 (2GHz Xeon)
- ▶ HyperThreading enabled
- ▶ 2-4 GB memory per machine
- ▶ Two 160GB IDE disks
- ▶ Gigabit Ethernet

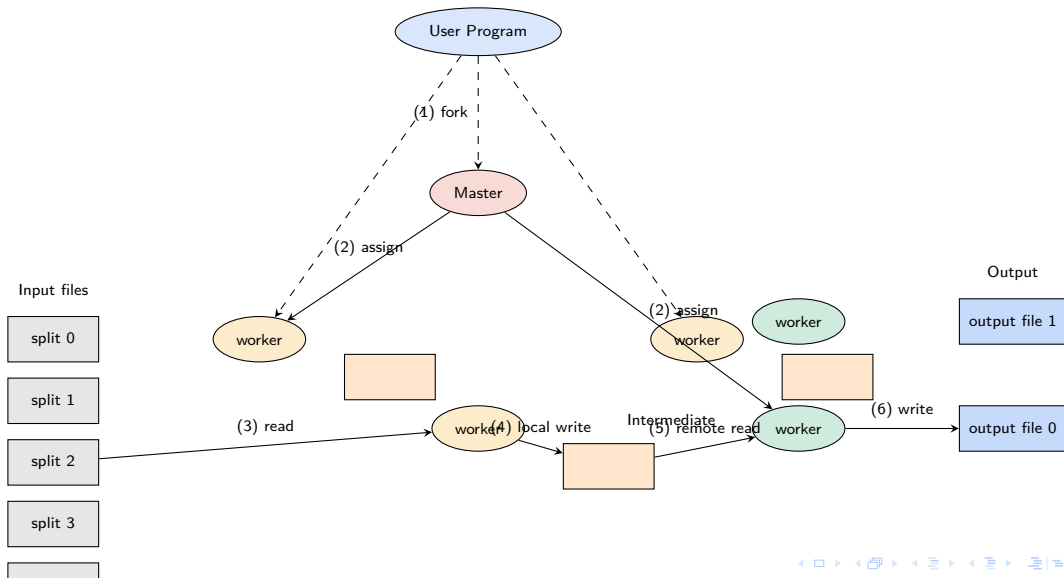
Environment:

- ▶ Hundreds/thousands of machines
- ▶ Two-level tree-switched network
- ▶ 100-200 Gbps aggregate bandwidth
- ▶ Machine failures are **common**
- ▶ Distributed file system (GFS)
- ▶ Job scheduling system

Key Insight

Network bandwidth is a **scarce resource** → Optimize for locality!

Execution Overview



Execution Steps (1/2)

Step 1: Split Input

- ▶ MapReduce library splits input files into M pieces (typically 16-64 MB)
- ▶ Starts up many copies of the program on cluster machines

Step 2: Master Assignment

- ▶ One copy is special: the **master**
- ▶ Rest are workers assigned work by the master
- ▶ Master assigns M map tasks and R reduce tasks to idle workers

Step 3: Map Task Execution

- ▶ Worker reads contents of corresponding input split
- ▶ Parses key/value pairs from input data
- ▶ Passes each pair to user-defined Map function
- ▶ Intermediate key/value pairs buffered in memory

Execution Steps (2/2)

Step 4: Write Intermediate Data

- ▶ Buffered pairs periodically written to local disk
- ▶ Partitioned into R regions by partitioning function
- ▶ Locations passed back to master

Step 5: Reduce Reads Intermediate Data

- ▶ Master notifies reduce worker about locations
- ▶ Reduce worker uses RPC to read buffered data from map workers' local disks
- ▶ Sorts intermediate data by key (groups occurrences of same key)
- ▶ External sort used if data too large for memory

Steps 6-7: Reduce and Complete

- ▶ Reduce worker iterates over sorted data
- ▶ For each unique key, passes key and values to Reduce function
- ▶ Output appended to final output file for this reduce partition
- ▶ When all tasks complete, master wakes up user program

Master Data Structures

State Tracking:

- ▶ For each map and reduce task:
 - ▶ State: *idle*, *in-progress*, or *completed*
 - ▶ Identity of worker machine (for non-idle tasks)

Location Propagation:

- ▶ Master is conduit for intermediate file regions
- ▶ For each completed map task, stores:
 - ▶ Locations of R intermediate file regions
 - ▶ Sizes of intermediate file regions
- ▶ Information pushed incrementally to workers with in-progress reduce tasks

Memory Overhead

$O(M + R)$ scheduling decisions and $O(M \times R)$ state in memory
(Constant factors are small: ≈ 1 byte per map/reduce task pair)

Fault Tolerance: Worker Failure

Detection:

- ▶ Master pings workers periodically
- ▶ No response in certain time → worker marked as failed

Recovery:

- ▶ Completed map tasks reset to *idle* state
- ▶ Map/reduce tasks in progress reset to *idle*
- ▶ Eligible for rescheduling on other workers

Why re-execute completed map tasks?

- ▶ Output stored on local disk of failed machine → **inaccessible**
- ▶ Completed reduce tasks don't need re-execution
- ▶ Their output stored in global file system

Resilience Example

Network maintenance caused 80 machines to become unreachable.
MapReduce simply re-executed work and continued to completion!

Fault Tolerance: Master Failure & Semantics

Master Failure:

- ▶ Master writes periodic checkpoints
- ▶ New copy can be started from last checkpoint
- ▶ Current implementation: abort if master fails (clients can retry)
- ▶ Single master makes failure unlikely

Semantics in Presence of Failures:

Deterministic Functions

When map and reduce operators are **deterministic**:

- ▶ Distributed implementation produces same output as non-faulting sequential execution
- ▶ Relies on atomic commits of map and reduce task outputs

Non-Deterministic Functions

Weaker but reasonable semantics:

- ▶ Output equivalent to some sequential execution

Locality Optimization

Key Observation

Network bandwidth is a **relatively scarce resource**

Strategy:

- ▶ Input data stored on local disks via GFS (Google File System)
- ▶ GFS divides files into 64 MB blocks
- ▶ Stores 3 copies of each block on different machines
- ▶ Master takes location information into account

Scheduling Policy:

1. **Best:** Schedule map task on machine containing replica of input data
2. **Next best:** Schedule near a replica (e.g., same network switch)

Result

When running large MapReduce operations on significant fraction of workers:

Most input data read locally → consumes **no network bandwidth**

Task Granularity

Subdividing:

- ▶ Map phase: M pieces
- ▶ Reduce phase: R pieces

Ideally: M and R should be much larger than number of worker machines

Benefits:

- ▶ Improves dynamic load balancing
- ▶ Speeds up recovery when worker fails
- ▶ Completed tasks spread across all other workers

Practical Bounds:

- ▶ Master makes $O(M + R)$ scheduling decisions
- ▶ Keeps $O(M \times R)$ state in memory

Typical Configuration at Google

$M = 200,000$, $R = 5,000$ using 2,000 worker machines

Each task: 16-64 MB of input data

Backup Tasks: Fighting Stragglers

Problem: Stragglers

A machine taking unusually long time to complete one of last few tasks

Causes of Stragglers:

- ▶ Bad disk (30 MB/s $\rightarrow$ 1 MB/s due to correctable errors)
- ▶ Cluster scheduler scheduled other tasks (competition for resources)
- ▶ Bug in machine initialization (e.g., processor caches disabled)

Solution: Backup Tasks

- ▶ When MapReduce operation close to completion
- ▶ Master schedules **backup executions** of remaining in-progress tasks
- ▶ Task marked complete when **either** primary **or** backup completes
- ▶ Tuned to increase resources by only a few percent

Impact

Sort program: **44% longer** to complete when backup tasks disabled!

Refinement 1: Partitioning Function

Default: Hash-based partitioning

- ▶ `hash(key) mod R`
- ▶ Results in fairly well-balanced partitions

Custom Partitioning:

- ▶ Sometimes useful to partition by other functions
- ▶ **Example:** Output keys are URLs
- ▶ Want all URLs from same host in same output file
- ▶ Solution: `hash(Hostname(urlkey)) mod R`

Benefits:

- ▶ Better data organization
- ▶ Enables efficient downstream processing
- ▶ Supports application-specific requirements

Refinement 2: Ordering Guarantees

Guarantee

Within a given partition, intermediate key/value pairs processed in **increasing key order**

Benefits:

- ▶ Easy to generate sorted output file per partition
- ▶ Useful when output format needs efficient random access by key
- ▶ Convenient for users who want sorted data

Use Cases:

- ▶ Building searchable indices
- ▶ Database-like access patterns
- ▶ Sorted data analysis

Refinement 3: Combiner Function

Problem

Significant repetition in intermediate keys produced by map tasks

Example: Word counting - many $\langle \text{the}, 1 \rangle$ records

Solution: Combiner Function

- ▶ Partial merging of data **before** sending over network
- ▶ Executed on each machine performing map task
- ▶ Typically same code as reduce function

Differences from Reduce:

- ▶ **Reduce:** Output written to final output file
- ▶ **Combiner:** Output written to intermediate file sent to reduce task

Impact

Partial combining **significantly speeds up** certain classes of MapReduce operations

Other Refinements

4. Input and Output Types

- ▶ Support for reading data in several formats (text, binary, database, etc.)
- ▶ Users can add custom input/output types via reader interface

5. Skipping Bad Records

- ▶ Detect records causing deterministic crashes
- ▶ Skip these records to make forward progress
- ▶ Useful when bugs in third-party libraries can't be fixed

6. Local Execution

- ▶ Alternative implementation for debugging
- ▶ Executes all work sequentially on local machine
- ▶ Facilitates debugging, profiling, small-scale testing

7. Status Information & Counters

Cluster Configuration

Test Environment:

- ▶ Approximately 1800 machines
- ▶ Dual-processor 2GHz Intel Xeon with HyperThreading
- ▶ 4GB memory per machine
- ▶ Two 160GB IDE disks per machine
- ▶ Gigabit Ethernet

Network:

- ▶ Two-level tree-shaped switched network
- ▶ $\approx 100\text{-}200$ Gbps aggregate bandwidth at root
- ▶ Round-trip time between any pair: < 1 millisecond

Conditions:

- ▶ Weekend afternoon execution
- ▶ CPUs, disks, network mostly idle
- ▶ 1-1.5GB reserved for other tasks

Benchmark 1: Grep

Task: Scan through 10^{10} 100-byte records, search for rare three-character pattern

Configuration:

- ▶ Input: 1 TB, split into ≈ 64 MB pieces ($M = 15,000$)
- ▶ Output: One file ($R = 1$)
- ▶ Pattern occurs in 92,337 records

Results:

- ▶ Peak input rate: **30+ GB/s** (1764 workers)
- ▶ Computation time: **150 seconds** (including ≈ 60 s startup overhead)
- ▶ Startup overhead due to:
 - ▶ Program propagation to all workers
 - ▶ GFS interactions to open 1000 input files
 - ▶ Locality optimization information gathering

Key Insight

Input rate gradually increases as more workers assigned, peaks, then drops as tasks complete

Benchmark 2: Sort (1/2)

Task: Sort 10^{10} 100-byte records (≈ 1 TB of data)

Configuration:

- ▶ Modeled after TeraSort benchmark
- ▶ Input split into 64MB pieces ($M = 15,000$)
- ▶ Output: 4000 files ($R = 4,000$)
- ▶ Output: 2 TB (2-way replicated GFS files)

Implementation:

- ▶ Map: Extract 10-byte sorting key, emit $\langle \text{key}, \text{line} \rangle$
- ▶ Reduce: Identity function (passes pairs unchanged)
- ▶ Less than 50 lines of user code!

Partitioning:

- ▶ Built-in knowledge of key distribution
- ▶ General sorting: pre-pass to sample keys and compute split-points

Benchmark 2: Sort (2/2)

Normal Execution Results:

- ▶ **Total time:** 891 seconds
- ▶ Input rate peaks: ≈ 13 GB/s
- ▶ Shuffle starts as first map completes
- ▶ Shuffling done: ≈ 600 seconds
- ▶ Write rate: 2-4 GB/s
- ▶ All writes finish: ≈ 850 seconds

Observations:

- ▶ Input rate $>$ shuffle rate $>$ output rate
- ▶ Locality optimization: most data read locally
- ▶ Shuffle rate $>$ output rate: 2 replicas written
- ▶ Comparable to TeraSort benchmark best result (1057 seconds)

Impact of Backup Tasks

Without backup tasks: **1283 seconds** (+44%)

Stragglers significantly impact completion time!

Performance: Machine Failures

Test: Intentionally killed 200 out of 1746 workers during sort

What Happened:

- ▶ Underlying cluster scheduler immediately restarted processes
- ▶ Deaths show up as negative input rate
- ▶ Previously completed map work disappeared
- ▶ Re-execution happened relatively quickly

Results:

- ▶ Total time: **933 seconds**
- ▶ Increase: Only **5%** over normal execution (891s)

Conclusion

MapReduce is highly resilient to machine failures!

System gracefully handles worker deaths without significant performance degradation.

MapReduce at Google

Timeline:

- ▶ First version: February 2003
- ▶ Significant enhancements: August 2003
- ▶ Locality optimization, dynamic load balancing, etc.

Growth:

- ▶ 0 instances in early 2003
- ▶ Almost 900 separate instances by September 2004
- ▶ Used across wide range of domains

Applications:

- | | |
|--|--------------------------------------|
| ▶ Large-scale machine learning | Zeitgeist) |
| ▶ Clustering problems (Google News, Froogle) | ▶ Property extraction from web pages |
| ▶ Popular query reports (Google | ▶ Large-scale graph computations |
| | ▶ Localized search data extraction |

Usage Statistics (August 2004)

Metric	Value
Number of jobs	29,423
Average job completion time	634 secs
Machine days used	79,186 days
Input data read	3,288 TB
Intermediate data produced	758 TB
Output data written	193 TB
Average worker machines per job	157
Average worker deaths per job	1.2
Average map tasks per job	3,351
Average reduce tasks per job	55
Unique map implementations	395
Unique reduce implementations	269
Unique map/reduce combinations	426

Large-Scale Indexing

Most Significant Use: Complete rewrite of production indexing system

Input: 20+ TB of crawled documents stored in GFS

Process: Sequence of 5-10 MapReduce operations

Benefits of Using MapReduce:

1. Simpler Code

- ▶ One phase: 3800 lines C++ → 700 lines with MapReduce
- ▶ Fault tolerance, distribution, parallelization hidden in library

2. Better Performance

- ▶ Can keep conceptually unrelated computations separate
- ▶ No need to mix together to avoid extra data passes

3. Easier to Operate

- ▶ Machine failures, slow machines, networking issues handled automatically
- ▶ Easy to improve performance: just add more machines!

Why MapReduce is Successful

1. Easy to Use

- ▶ No parallel/distributed systems experience needed
- ▶ Hides complexity of parallelization, fault-tolerance, locality, load balancing
- ▶ Simple program can run on 1000 machines in half an hour

2. Widely Applicable

- ▶ Large variety of problems expressible as MapReduce
- ▶ Used for web search, sorting, data mining, machine learning, and more

3. Scalable Implementation

- ▶ Scales to thousands of machines
- ▶ Efficient use of machine resources
- ▶ Suitable for Google's large computational problems

Development Impact

Greatly speeds up development and prototyping cycle

Allows programmers to exploit large amounts of resources easily

Key Lessons Learned

1. Restricting the Programming Model

- ▶ Makes it easy to parallelize and distribute computations
- ▶ Enables fault-tolerance through re-execution
- ▶ Simplicity is powerful!

2. Network Bandwidth is Scarce

- ▶ Many optimizations target reducing network usage
- ▶ Locality optimization: read from local disks
- ▶ Single copy of intermediate data to local disk saves bandwidth

3. Redundant Execution is Valuable

- ▶ Reduces impact of slow machines (stragglers)
- ▶ Handles machine failures gracefully
- ▶ Handles data loss effectively

Impact and Legacy

Historical Impact

This paper (OSDI 2004) became one of the most influential papers in distributed systems and big data processing

Inspired Numerous Systems:

- ▶ **Apache Hadoop** (2006) - Open-source MapReduce implementation
- ▶ **Apache Spark** - Next-generation data processing
- ▶ **Apache Flink, Apache Beam** - Stream processing frameworks
- ▶ Countless other distributed data processing systems

Paradigm Shift:

- ▶ Democratized large-scale data processing
- ▶ Made distributed computing accessible to non-experts
- ▶ Enabled the "Big Data" revolution
- ▶ Foundation for modern data engineering practices

Summary

MapReduce in a Nutshell

A programming model and implementation for processing large datasets on clusters of commodity machines

Key Characteristics:

- ▶ Simple programming interface (Map and Reduce functions)
- ▶ Automatic parallelization and distribution
- ▶ Fault tolerance via re-execution
- ▶ Locality optimization for performance
- ▶ Scalable to thousands of machines

Why It Matters:

- ▶ Hides complexity of distributed systems
- ▶ Allows programmers to focus on logic, not infrastructure
- ▶ Proven at massive scale at Google
- ▶ Influenced entire generation of big data systems

Questions to Consider

1. What types of problems are *not* well-suited for MapReduce?
2. How does MapReduce compare to modern frameworks like Apache Spark?
3. What are the trade-offs between simplicity and flexibility in the MapReduce model?
4. How would you extend MapReduce to support iterative algorithms?
5. What role does MapReduce play in today's data processing landscape?